APM 2025-2026 : Contrôle Continu 1 - Corrigé

18 octobre 2026

Table des matières

I Questions de cours **2**
Q.1 cudaMallocHost, cudaMalloc et malloc . . . 2
Q.2 Accès coalescé . . . 2
Q.3 Indice global : blocs 3D et grille 2D . . . 3
Q.4 Attribut de données inexistant . . . 3
Q.5 Équivalent OpenACC d'une région de données OpenMP . . . 3

II Succession de kernels **4**
Q.6 Dépendances nécessaires avec des événements . . . 4

III Safe/Unsafe : expliciter les hypothèses **6**
Q.7 tmp est un pointeur hôte pinned : réponse conditionnelle . . . 6
Q.8 Octets utilisés comme déplacement de pointeur int . . . 6
Q.9 Ordre du stream par défaut et disponibilité pour le CPU . . . 7
Q.10 Deux erreurs indépendantes : bornes et dépendances inter-stream . . . 8

IV Réduction somme en CUDA **8**
Q.11 Réduction partagée et deux lancements . . . 8

V Adapter la préparation à ce sujet **10**

Le sujet comporte 11 questions et indique une durée de 3 heures, sans barème visible.

Sujet pour s'entraîner • Explications chinoises • Sources • Dossier général de révision

I Questions de cours

Q.1 cudaMallocHost, cudaMalloc et malloc

Question : expliquer les allocations et leurs différences.

Fonction	Emplacement et propriété usuels	Libération
`malloc(bytes)`	Mémoire hôte ordinaire, généralement pageable, destinée au CPU	`free(ptr)`
`cudaMallocHost(&ptr, bytes)`	Mémoire hôte verrouillée en pages, ou pinned, utilisable pour les transferts CUDA	`cudaFreeHost(ptr)`
`cudaMalloc(&ptr, bytes)`	Mémoire device du GPU dans le modèle étudié	`cudaFree(ptr)`

`cudaMallocHost` écrit le résultat dans la variable pointeur du demandeur, d'où `&ptr` ; la taille est en octets. L'allocation n'initialise pas son contenu. La mémoire pinned convient aux transferts H2D/D2H asynchrones et évite le staging habituellement nécessaire avec la mémoire pageable. Elle permet d'envisager un recouvrement avec le calcul, sans garantir un gain dans tout programme.

Lien avec Q.7 : l'accès direct d'un kernel à une allocation hôte dépend du mapping et des capacités du dispositif. La mémoire pinned n'est pas universellement inaccessible au GPU ; elle ne se confond pas non plus avec une allocation managed par `cudaMallocManaged`. Q.7 détaille cette distinction. Verrouiller une quantité excessive de mémoire peut aussi pénaliser le système.

Réponse courte : `cudaMallocHost` alloue une mémoire hôte verrouillée en pages, adaptée aux transferts CUDA asynchrones. `cudaMalloc` alloue une mémoire device, tandis que `malloc` alloue une mémoire hôte ordinaire. Les fonctions de libération correspondantes sont `cudaFreeHost`, `cudaFree` et `free`.

Révision : sections 2.4 et 2.11.

Q.2 Accès coalescé

Question : définir un accès coalescé.

Lorsque les adresses de mémoire globale demandées par les threads actifs d'un warp sont proches et correctement alignées, le matériel peut généralement les servir avec peu de transactions. Un motif favorable classique associe les lanes voisines aux éléments voisins :

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
value = a[i];
```

Un accès `a[i * stride]` pour un grand `stride` disperse les adresses sur davantage de segments. Le nombre exact de transactions dépend aussi du type, de l'alignement, des threads actifs et de l'architecture : 32 threads n'impliquent pas systématiquement une transaction unique. La coalescence concerne les adresses d'un warp et n'est pas une synchronisation.

Réponse courte : Un accès coalescé regroupe les accès mémoire des threads d'un warp en un petit nombre de transactions. Des threads voisins accédant à des éléments contigus et correctement alignés favorisent cette propriété.

Révision : section 2.7.

Q.3 Indice global : blocs 3D et grille 2D

Question : construire un numéro unique en annonçant la convention.

On fait varier x le plus vite, puis y, puis z. On numérote d'abord les blocs, puis on range consécutivement tous les threads de chaque bloc :

$$\begin{aligned} b &= \text{blockIdx.x} + \text{GRID_X}\,\text{blockIdx.y}, \\ t &= \text{threadIdx.x} + \text{BLOCK_X}(\text{threadIdx.y} + \text{BLOCK_Y}\,\text{threadIdx.z}), \\ B &= \text{BLOCK_X}\,\text{BLOCK_Y}\,\text{BLOCK_Z}, \\ g &= bB + t. \end{aligned}$$

```
size_t block_id = (size_t)blockIdx.y * gridDim.x + blockIdx.x;
size_t threads_per_block = (size_t)blockDim.x * blockDim.y * blockDim.z;
size_t local_id = threadIdx.x + (size_t)blockDim.x *
                  (threadIdx.y + (size_t)blockDim.y * threadIdx.z);
size_t global_id = block_id * threads_per_block + local_id;
```

`GRID_Y` fixe le nombre total de blocs et la borne supérieure, sans devoir figurer séparément dans la formule du bloc courant. Le total vaut `GRID_X*GRID_Y*B`, et les indices vont de 0 à ce total moins 1. La grille étant 2D, aucun terme `blockIdx.z` n'est ajouté.

Exemple : `grid=(3,2)`, `block=(4,2,2)`, `blockIdx=(1,1)`, `threadIdx=(2,1,1)` donnent `b=4`, `B=16`, `t=14`, `g=78`. L'adresse d'un tableau spatial 3D demanderait une autre linéarisation des coordonnées des données.

Révision : sections 1.10, 1.11 et 4.2.

Q.4 Attribut de données inexistant

Réponse : proposition 3, `__local__`.

CUDA possède un espace de mémoire locale, mais pas l'attribut de déclaration C/C++ `__local__` proposé ici. Les variables privées ordinaires peuvent être placées en registres ou en mémoire locale selon leur usage et le compilateur.

`__device__` peut qualifier une variable device aussi bien qu'une fonction device : son usage fréquent pour des fonctions ne le rend pas incorrect dans cette liste. `__shared__` et `__constant__` qualifient les variables des espaces correspondants. Ne pas confondre avec `__local` en OpenCL.

Révision : sections 2.1 et 2.3.

Q.5 Équivalent OpenACC d'une région de données OpenMP

Réponse :

```
#pragma acc data copyin(a[0:8])
{
    /* Region using a on the accelerator. */
}
```

La région structurée `target data map(to: a[0:8])` correspond ici à une région OpenACC `data copyin`. Dans le cas usuel d'un dispositif séparé, sans mapping préexistant, les

données sont créées et copiées à l'entrée, sans rapatriement des modifications à la sortie. `0:8` signifie huit éléments à partir de l'indice zéro.

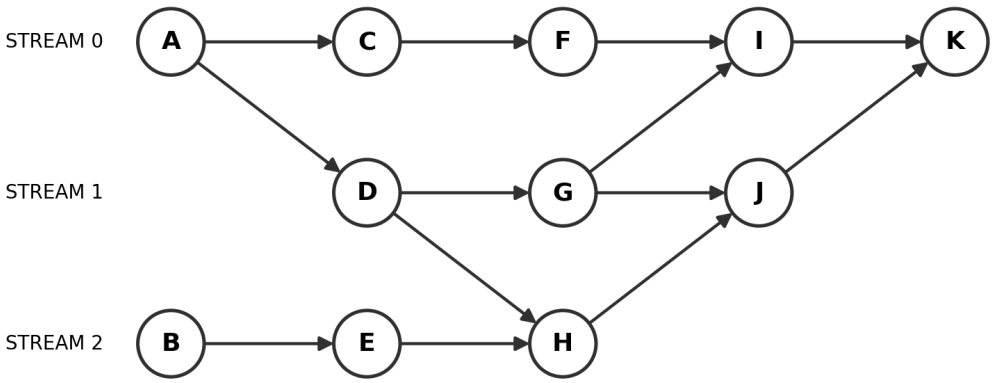
Une région `data` gère les données, sans effectuer elle-même un calcul parallèle ; une directive de calcul reste nécessaire à l'intérieur. `enter data copyin` introduit une durée de vie non structurée et ne constitue pas un remplacement identique sans explication.

Référence : guide officiel OpenACC. La section 3.13 du dossier général traite OpenMP Target ; cette réponse ajoute la correspondance OpenACC.

II Succession de kernels

Q.6 Dépendances nécessaires avec des événements

Question : soumettre les onze kernels sur trois streams et synchroniser seulement ce que demande le graphe.



L'ordre dans un même stream est déjà assuré :

- `s[0]` : A → C → F → I → K.
- `s[1]` : D → G → J.
- `s[2]` : B → E → H.

Il reste cinq dépendances entre streams : **A → D, D → H, G → I, H → J, J → K**. L'ordre des appels CPU ne prouve pas un ordre d'achèvement GPU entre streams distincts.

Le code suivant montre uniquement les soumissions et synchronisations. Il suppose les kernels, leurs arguments et configurations définis, ainsi que le succès des API. Dans une copie, `A<<<...>>>(...)` peut représenter les arguments correspondants ; ils sont uniformément omis ici.

```
cudaStream_t s[3];
for (int i = 0; i < 3; ++i)
    cudaStreamCreateWithFlags(&s[i], cudaStreamNonBlocking);
cudaEvent_t eA, eD, eG, eH, eJ;
cudaEventCreateWithFlags(&eA, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eD, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eG, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eH, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eJ, cudaEventDisableTiming);

A<<<grid, block, 0, s[0]>>>();
cudaEventRecord(eA, s[0]);
B<<<grid, block, 0, s[2]>>>();
C<<<grid, block, 0, s[0]>>>();
F<<<grid, block, 0, s[0]>>>();

cudaStreamWaitEvent(s[1], eA, 0);
D<<<grid, block, 0, s[1]>>>();
cudaEventRecord(eD, s[1]);
G<<<grid, block, 0, s[1]>>>();
cudaEventRecord(eG, s[1]);

E<<<grid, block, 0, s[2]>>>();
cudaStreamWaitEvent(s[2], eD, 0);
H<<<grid, block, 0, s[2]>>>();
cudaEventRecord(eH, s[2]);

cudaStreamWaitEvent(s[0], eG, 0);
I<<<grid, block, 0, s[0]>>>();
cudaStreamWaitEvent(s[1], eH, 0);
J<<<grid, block, 0, s[1]>>>();
cudaEventRecord(eJ, s[1]);
cudaStreamWaitEvent(s[0], eJ, 0);
K<<<grid, block, 0, s[0]>>>();

// Before the CPU consumes results or resources are released:
cudaStreamSynchronize(s[0]);
for (int i = 0; i < 3; ++i) cudaStreamDestroy(s[i]);
cudaEventDestroy(eA);
cudaEventDestroy(eD);
cudaEventDestroy(eG);
cudaEventDestroy(eH);
cudaEventDestroy(eJ);
```

Pourquoi ces attentes ? Elles précèdent le consommateur : D attend A, H attend D, I attend G, J attend H et K attend J. Chaque événement est enregistré après son producteur. Dans cet exemple, chaque record est soumis par le CPU avant le wait associé, évitant d'attendre un événement non encore enregistré. `cudaStreamWaitEvent` ajoute une dépendance aux files GPU, sans arrêter le CPU à chaque arête.

Sens du minimum : en conservant les possibilités de parallélisme du graphe, sans ordre global supplémentaire, aucune des cinq arêtes inter-stream n'est déductible des autres. Avec un événement après chaque producteur et une attente avant chaque consommateur, cinq waits les couvrent exactement. Réduire seulement le nombre d'appels d'attente côté hôte en sérialisant davantage peut ajouter des dépendances absentes du sujet.

K succède à tous les autres nœuds. Attendre sa fin dans `s[0]` couvre donc tous les kernels du graphe ; une synchronisation globale après chaque producteur est inutile. Des copies ou utilisateurs de résultats extérieurs au graphe demanderaient une analyse supplémentaire.

Références : événements NVIDIA, attente d'événement dans un stream. Révision : sections 2.11 et 4.13.

III Safe/Unsafe : expliciter les hypothèses

Les fonctions kernel, certaines déclarations périphériques et le contrôle complet des erreurs ne sont pas fournis. On suppose `N>0`, des tailles sans débordement arithmétique, des lancements valides et des kernels limités aux zones annoncées. L'absence de leur corps ne permet pas d'inventer les résultats. Les fragments originaux sont conservés dans le sujet séparé.

Q.7 tmp est un pointeur hôte pinned : réponse conditionnelle

Question : un stream enchaîne H2D, kernel, D2H et attente, puis le CPU affiche. Le kernel reçoit aussi `tmp`, alloué par `cudaMallocHost`.

Ordre déjà établi : toutes les opérations utilisent `stream[0]` et le `cudaStreamSynchronize` suit le D2H. L'affichage attend donc bien la copie. Il ne manque pas ici une attente D2H.

À vérifier : l'accessibilité device de `tmp` et son initialisation. Dans un modèle pédagogique simplifié à mémoires séparées, passer directement un pointeur hôte peut conduire à « unsafe ». Ce n'est pas une conclusion universelle : sur un dispositif supportant UVA, NVIDIA prévoit l'accès direct aux allocations ordinaires `cudaMallocHost` par le même pointeur.

La photo seule ne permet donc ni de conclure inconditionnellement unsafe, ni de prouver safe. Si cet accès est supporté, si le kernel écrit correctement son espace de travail avant de le lire et reste dans les bornes, l'ordre des opérations est sûr. Si le kernel lit `tmp` non initialisé ou si l'environnement interdit l'accès direct, il est incorrect. Une réponse argumentée doit annoncer cette hypothèse.

Correction portable : pour un espace de travail device, allouer avec `cudaMalloc` et initialiser avant toute lecture. Pour de la mémoire hôte mappée, vérifier explicitement la capacité et le pointeur device valide, puis ordonner les accès CPU/GPU.

```
int *d_tmp = nullptr;
size_t bytes = (size_t)N * sizeof(int);
cudaMalloc((void **)&d_tmp, bytes);
// Only needed if the kernel reads the initial scratch values:
cudaMemsetAsync(d_tmp, 0, bytes, stream[0]);
kernel<<<grid, block, 0, stream[0]>>>(dtab, d_tmp, N);
// Queue the D2H copy, wait for the stream, then cudaFree(d_tmp).
```

Cet extrait remplace l'allocation de `tmp` et l'argument du kernel ; il n'ajoute pas un second calcul au programme original.

Référence : mapping automatique des allocations hôte dans UVA. Révision : sections 2.11 et 3.5.

Q.8 Octets utilisés comme déplacement de pointeur int

Conclusion : unsafe. Le problème principal est l'unité du déplacement.

`size=N*sizeof(int)` est en octets, mais `dtab + size` avance de `size` éléments int. Avec l'exemple usuel CUDA `sizeof(int)=4` :

Opération	Intervalle d'éléments attendu	Intervalle réel du code
stream 0, N éléments	`[0,N)`	`[0,N)`
stream 1, 2N éléments	`[N,3N)`	`[4N,6N)`
stream 2, 3N éléments	`[3N,6N)`	`[12N,15N)`

L'allocation device contient `6N` int et celle de l'hôte `7N` int : la troisième zone déborde. La deuxième reste ici dans l'allocation, mais ne correspond pas au deuxième segment

contigu attendu. Il faut corriger **les pointeurs des copies et des kernels**, pas seulement les tailles de transfert.

```
size_t bytes = (size_t)N * sizeof(int);
cudaMemcpyAsync(dtab, htab, bytes,
                cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + N, htab + N, 2 * bytes,
                cudaMemcpyHostToDevice, stream[1]);
cudaMemcpyAsync(dtab + 3 * N, htab + 3 * N, 3 * bytes,
                cudaMemcpyHostToDevice, stream[2]);
kernel<<<grid0, block0, 0, stream[0]>>>(dtab, N);
kernel<<<grid1, block1, 0, stream[1]>>>(dtab + N, 2 * N);
kernel<<<grid2, block2, 0, stream[2]>>>(dtab + 3 * N, 3 * N);
cudaMemcpyAsync(htab, dtab, bytes,
                cudaMemcpyDeviceToHost, stream[0]);
cudaMemcpyAsync(htab + N, dtab + N, 2 * bytes,
                cudaMemcpyDeviceToHost, stream[1]);
cudaMemcpyAsync(htab + 3 * N, dtab + 3 * N, 3 * bytes,
                cudaMemcpyDeviceToHost, stream[2]);
for (int s = 0; s < 3; ++s) cudaStreamSynchronize(stream[s]);
```

Conserver les allocations et le memset du sujet ; remplacer ses copies, lancements et attentes par ce fragment. Les zones sont disjointes, donc aucun événement inter-stream n'est requis. Le sujet attend déjà chaque D2H dans son stream : dire qu'il n'attend que le troisième stream serait incorrect. Regrouper les attentes à la fin clarifie simplement le code.

L'affichage de `7N` éléments ne dépasse pas le buffer hôte alloué à cette taille. Toutefois, seuls les `6N` premiers représentent les résultats GPU ; les `N` derniers conservent zéro. Afficher `6N` éléments si seuls les résultats sont souhaités.

Révision : sections 2.4 et 4.13.

Q.9 Ordre du stream par défaut et disponibilité pour le CPU

Conclusion : unsafe, avec attente hôte manquante et entrée non initialisée.

Dans l'interface CUDA C++, l'argument stream omis des deux `cudaMemcpyAsync` vaut zéro. Le kernel utilise aussi ce stream par défaut : l'ordre GPU H2D → kernel → D2H est assuré.

La copie D2H asynchrone vers le buffer pinned ne garantit pas la disponibilité des données au retour de l'appel. Le CPU lit et modifie immédiatement `htab`, avec un risque de conflit avec la copie en cours. De plus, le buffer n'est pas initialisé avant H2D : un kernel lisant ces valeurs ne reçoit pas une entrée définie.

Initialiser l'entrée, soumettre H2D, kernel et D2H, puis attendre avant le décalage CPU. On peut attendre le stream par défaut correspondant, ou employer explicitement le même stream pour toutes les opérations.

```
// Initialize htab before the H2D copy.
memset(htab, 0, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice);
kernel<<<grid, block>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost);
cudaStreamSynchronize(0);
for (int i = 0; i < N - 1; ++i) htab[i] = htab[i + 1];
htab[N - 1] = 0;
```

Ce fragment remplace la séquence concernée et place l'initialisation avant H2D. Si une autre partie du programme fournit déjà la bonne entrée, la conserver au lieu de la remettre à zéro. Les règles particulières du stream par défaut ne remplacent pas l'attente du D2H par le CPU.

Référence : comportement de synchronisation des API NVIDIA. Révision : sections 2.11 et 4.13.

Q.10 Deux erreurs indépendantes : bornes et dépendances inter-stream

Conclusion : unsafe. Corriger les déplacements ne suffit pas.

Première erreur : unité des pointeurs. `sz_in_bytes` est en octets, tandis que `dtab + sz_in_bytes` avance en int. Pour des int de 4 octets, le deuxième segment commence à l'élément `4N`, au-delà de l'allocation de `2N` éléments. Utiliser `dtab+N` et `htab+N`.

Deuxième erreur : les kernels traitent la zone de l'autre stream. Après correction des pointeurs, on conserve sinon :

- stream 0 : H2D première zone → kernel deuxième zone → D2H première zone.
- stream 1 : H2D deuxième zone → kernel première zone → D2H deuxième zone.

Le kernel de la première zone, dans stream 1, dépend du H2D du stream 0 ; son D2H dans stream 0 dépend du kernel du stream 1. La deuxième zone présente les dépendances symétriques, non exprimées dans le code.

Le premier `cudaStreamSynchronize(stream[0])` est situé **après** le D2H de la première zone. Attendre des opérations déjà soumises ne les réordonne pas rétroactivement pour corriger leur dépendance avec stream 1. Les attentes finales n'effacent pas une course déjà survenue. Un résultat correct dans un essai ne prouve pas la sûreté.

Correction directe : un stream prend en charge toute la chaîne d'une même zone.

```
size_t bytes = (size_t)N * sizeof(int);
for (int s = 0; s < 2; ++s) {
    size_t offset = (size_t)s * N;
    cudaMemcpyAsync(dtab + offset, htab + offset, bytes,
                    cudaMemcpyHostToDevice, stream[s]);
    kernel<<<grid, block, 0, stream[s]>>>(dtab + offset, N);
    cudaMemcpyAsync(htab + offset, dtab + offset, bytes,
                    cudaMemcpyDeviceToHost, stream[s]);
}
for (int s = 0; s < 2; ++s) cudaStreamSynchronize(stream[s]);
for (int i = 0; i < 2 * N; ++i) printf("%d", htab[i]);
```

Conserver les allocations de `2N` et le memset du sujet ; remplacer les copies, kernels et attentes. Si la répartition croisée doit être conservée, enregistrer des événements après chaque producteur et les attendre avant les consommateurs correspondants. Garder un stream par zone simplifie la preuve.

Révision : sections 2.11 et 4.13.

IV Réduction somme en CUDA

Q.11 Réduction partagée et deux lancements

Question : les données sont déjà sur le GPU. Un thread lit un élément, un bloc calcule une somme partielle, puis un second kernel réduit ces sommes.

La première phase utilise `G` blocs de `B` threads, avec `N=G*B` et `G`, `B` puissances de deux. Chaque thread écrit son entrée en mémoire partagée ; les étapes successives combinent les valeurs avec les pas `B/2, B/4, ..., 1`.

```cpp
__global__ void reduce_kernel(float *in, float *out)
{
    extern __shared__ float partial[];
    unsigned int t = threadIdx.x;
    size_t i = (size_t)blockIdx.x * blockDim.x + t;
    partial[t] = in[i];
    __syncthreads();

    for (unsigned int stride = blockDim.x / 2;
         stride > 0; stride /= 2) {
        if (t < stride)
            partial[t] += partial[t + stride];
        __syncthreads();
    }
    if (t == 0) out[blockIdx.x] = partial[0];
}
```

La barrière est hors du `if (t < stride)` afin que tous les threads du bloc la rencontrent. Chaque étape dépend de la précédente ; ses barrières ne se suppriment pas arbitrairement. Chaque bloc écrit une case distincte, sans addition atomique inter-bloc.

```cpp
// tab is already allocated and initialized on the GPU.
// G and B are positive powers of two and N == G * B.
// Both B and G must fit the device's per-block resource limits.
float *block_sums = nullptr;
float *total = nullptr;
cudaMalloc((void **)&block_sums, (size_t)G * sizeof(float));
cudaMalloc((void **)&total, sizeof(float));

reduce_kernel<<<G, B, (size_t)B * sizeof(float)>>>(tab, block_sums);
cudaGetLastError();
reduce_kernel<<<1, G, (size_t)G * sizeof(float)>>>(block_sums, total);
cudaGetLastError();

float host_sum = 0.0f;
cudaMemcpy(&host_sum, total, sizeof(float), cudaMemcpyDeviceToHost);
cudaFree(total);
cudaFree(block_sums);
```

Le troisième paramètre du lancement est la taille, en octets, de mémoire partagée dynamique par bloc. Le deuxième kernel utilise un bloc de `G` threads pour les `G` sommes partielles. Les deux lancements dans le même stream sont ordonnés : aucune synchronisation CPU intermédiaire n'est nécessaire. Le D2H synchrone final rend la somme disponible à son retour. Un vrai programme doit vérifier les erreurs des API et des lancements ; les fragments constituent ici une réponse écrite.

Condition matérielle implicite : une puissance de deux ne garantit pas qu'un bloc de `G` threads soit légal. `G` et `B` doivent respecter les limites de threads et de mémoire partagée du dispositif. Si `G` est trop grand, employer plusieurs niveaux ou modifier la deuxième phase pour faire accumuler plusieurs sommes partielles par thread. Le lancement `<<<1,G>>>` ne convient pas sans limite à toute taille de tableau.

Exemple : `N=16`, `G=4`, `B=4`, entrée `1...16`. La première phase produit `[10,26,42,58]`, puis la seconde donne `136`. Le changement d'ordre des additions flottantes n'assure pas l'égalité bit à bit avec une somme séquentielle ; vérifier les valeurs finies et une tolérance absolue/relative appropriée.

V Adapter la préparation à ce sujet

Types de questions posées dans ce sujet. La durée de trois heures est celle du sujet 2025-2026.

Travail prioritaire	Questions	Entrée de révision
Emplacement des données, pinned et mapped	Q1, Q7	Sections 2.4, 2.11 et 3.5
Coalescence et linéarisation des threads	Q2, Q3	Sections 1.10, 1.11, 2.7 et 4.2
Attributs CUDA et directives	Q4, Q5	Sections 2.1, 2.3 et 3.13 ; Q5 ci-dessus
Dessiner les dépendances et utiliser les events	Q6	Sections 2.11 et 4.13 ; Q6 ci-dessus
Vérifier unités, intervalles et disponibilité	Q7-Q10	Partie III ; justifier plutôt que mémoriser les verdicts
Réduction arborescente en mémoire partagée	Q11	Partie IV

Faire d’ abord le sujet français sans consulter les réponses. Pour chaque fragment, poser quatre questions : où réside le pointeur ? Quelles sont les unités de la taille et du déplacement ? Le consommateur attend-il son producteur ? Quand le CPU peut-il lire le résultat ? Réécrire enfin Q6 et Q11 en justifiant chaque dépendance et chaque barrière.